

Experiment 2

Dynamic Memory Allocation, Reallocation and Deallocation in C

Name	Harsh Sen
Program	B.Tech CSE (AI & ML)
Experiment	Exp 2 - Dynamic Memory Allocation, Reallocation, Deallocation
Tools used	gcc 13.3.0, AddressSanitizer (ASan), Ubuntu 24.04

1. Aim

To write a C program that allocates a dynamic array using **malloc()**, resizes it using **realloc()**, and releases it using **free()**, while printing the pointer value and allocated size at each stage. To deliberately introduce and study two common dynamic-memory errors — (a) an unchecked allocation return value leading to a **NULL pointer dereference**, and (b) a **use-after-free** through a dangling pointer — and to capture their diagnostics using AddressSanitizer (ASan). Finally, to write a corrected version of the program that always checks return values, initialises memory after allocation, and sets the pointer to NULL immediately after **free()**.

2. Background

C performs no automatic bounds checking or lifetime tracking on heap memory obtained through **malloc()**, **realloc()**, and **free()**. Two of the most common classes of memory-safety bugs that result from this are: (1) failing to check whether an allocation succeeded before using the returned pointer, which can lead to a NULL pointer dereference when memory is exhausted; and (2) continuing to use a pointer after the memory it refers to has been freed (a "dangling pointer"), which can lead to a use-after-free (UAF). Both are undefined behaviour in the C standard and are frequently exploitable in real software. AddressSanitizer (ASan) is a compiler-instrumentation tool (enabled with `-fsanitize=address`) that inserts runtime checks around heap allocations, so that such invalid accesses are caught immediately and reported with a precise stack trace, instead of silently corrupting memory or crashing far away from the root cause.

3. Part A - Buggy Version 1: Unchecked malloc() -> NULL Pointer Dereference

This program deliberately requests an unreasonably large block of memory (SIZE_MAX/2 ints), which no real system can satisfy, so that malloc() genuinely returns NULL. The program then dereferences this pointer without checking it first.

3.1 Code listing — buggy_null.c

```
/*
 * Experiment 2 - Bug (a): Unchecked malloc() return value
 * -> NULL pointer dereference
 *
 * We deliberately request an absurdly large block (larger than any
 * real system can supply) so malloc() genuinely returns NULL, then
 * we use the pointer WITHOUT checking it, causing a NULL dereference.
 *
 * Compile with ASan:
 * gcc -fsanitize=address -g -O0 -o buggy_null buggy_null.c
 */

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* Ask for an impossible amount of memory to force malloc() to fail */
    size_t huge_n = (size_t)-1 / 2; /* ~ SIZE_MAX/2 elements */
    int *arr = (int *) malloc(huge_n * sizeof(int));

    printf("Requested elements : %zu\n", huge_n);
    printf("Pointer value      : %p\n", (void *)arr); /* will print (nil) */
    fflush(stdout);

    /* BUG: no NULL check here before using arr */
    printf("Writing to arr[0]...\n");
    fflush(stdout);
    arr[0] = 42; /* NULL POINTER DEREFERENCE */

    printf("This line is never reached.\n");
    return 0;
}
```

3.2 Compilation

```
$ gcc -fsanitize=address -g -O0 -o buggy_null buggy_null.c
```

3.3 Execution and ASan diagnostic output

ASan's allocator normally refuses (aborts on) absurdly large size requests before even attempting them; to force a genuine NULL return from malloc() (as an unchecked real-world out-of-memory condition would), the environment variable ASAN_OPTIONS=allocator_may_return_null=1 was set.

```
$ ASAN_OPTIONS=allocator_may_return_null=1 ./buggy_null

==499==WARNING: AddressSanitizer failed to allocate 0xfffffffffffffffc bytes
Requested elements : 9223372036854775807
Pointer value      : (nil)
Writing to arr[0]...
AddressSanitizer:DEADLYSIGNAL
=====
==499==ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000 (pc 0x556aa15a237d bp
0x7ffd76239580 sp 0x7ffd76239570 T0)
==499==The signal is caused by a WRITE memory access.
==499==Hint: address points to the zero page.
#0 0x556aa15a237d in main /home/claude/exp2/buggy_null.c:28
#1 0x7f727502a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#2 0x7f727502a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#3 0x556aa15a21a4 in _start (/home/claude/exp2/buggy_null+0x11a4) (BuildId:
d43815ae0c1ec9f284d492139135c1beaf421bcd)

AddressSanitizer can not provide additional info.
```

```
SUMMARY: AddressSanitizer: SEGV /home/claude/exp2/buggy_null.c:28 in main
==499==ABORTING
EXIT: 1
```

3.4 Explanation of this error case

`malloc()` was asked for far more memory than the system can provide, so it returned `NULL`. Because the return value was never checked, the program proceeded to execute `arr[0] = 42;`, which is equivalent to writing to address `0x0`. This triggers a hardware segmentation fault (`SIGSEGV`). ASan intercepts the signal and reports "SEGV on unknown address `0x000000000000`", together with the exact source line (`buggy_null.c:28`) and the call stack, confirming the fault is a `NULL`-pointer write.

3.5 Security impact

On most modern operating systems the zero page is unmapped, so a `NULL` dereference is usually 'only' a crash (a denial-of-service). However, unchecked-allocation bugs are still a genuine security concern: (i) an attacker who can control the requested size (e.g. via an untrusted length field) can deliberately force allocation failure to crash a service (DoS) at a moment of their choosing; (ii) in constrained or embedded environments the low address range can sometimes be mapped, turning what looks like a `NULL` dereference into an exploitable arbitrary read/write; and (iii) in a large program a single unchecked allocation deep in a library can bring down an entire process, which is disproportionate to the small coding omission (a missing `'if (ptr == NULL)'`) that caused it. Systematic input validation and return-value checking on every allocation is the standard mitigation.

4. Part B - Buggy Version 2: Use-After-Free via a Dangling Pointer

This program performs a normal malloc -> realloc -> free lifecycle, printing the pointer value and size at each stage as required, but then deliberately continues to read and write through the pointer after free() has already released the memory.

4.1 Code listing — buggy_uaf.c

```
/*
 * Experiment 2 - Bug (b): Use-after-free via a dangling pointer
 *
 * Compile with ASan:
 * gcc -fsanitize=address -g -O0 -o buggy_uaf buggy_uaf.c
 */

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;

    /* ---- Stage 1: malloc ---- */
    int *arr = (int *) malloc(n * sizeof(int));
    if (arr == NULL) {
        fprintf(stderr, "malloc failed\n");
        return 1;
    }
    printf("Stage 1 (malloc):\n");
    printf("  Pointer value   : %p\n", (void *)arr);
    printf("  Allocated size : %zu bytes (%d ints)\n\n", n * sizeof(int), n);
    fflush(stdout);

    for (int i = 0; i < n; i++) arr[i] = i + 1;

    /* ---- Stage 2: realloc ---- */
    int new_n = 10;
    int *tmp = (int *) realloc(arr, new_n * sizeof(int));
    if (tmp == NULL) {
        fprintf(stderr, "realloc failed\n");
        free(arr);
        return 1;
    }
    arr = tmp;
    printf("Stage 2 (realloc):\n");
    printf("  Pointer value   : %p\n", (void *)arr);
    printf("  Allocated size : %zu bytes (%d ints)\n\n", new_n * sizeof(int), new_n);
    fflush(stdout);

    for (int i = n; i < new_n; i++) arr[i] = (i + 1) * 10;

    /* ---- Stage 3: free ---- */
    free(arr);
    printf("Stage 3 (free):\n");
    printf("  Pointer value freed : %p\n", (void *)arr);
    printf("  Memory returned to heap allocator.\n\n");
    fflush(stdout);

    /* BUG: 'arr' is now a dangling pointer. We deliberately keep using it. */
    printf("Demonstrating use-after-free:\n");
    printf("  Reading arr[0] after free...\n");
    fflush(stdout);
    printf("  arr[0] = %d\n", arr[0]);    /* USE AFTER FREE (read) */

    printf("  Writing arr[0] after free...\n");
    arr[0] = 999;               /* USE AFTER FREE (write) */

    return 0;
}
```

4.2 Compilation

```
$ gcc -fsanitize=address -g -O0 -o buggy_uaf buggy_uaf.c
```

4.3 Execution and ASan diagnostic output

```
$ ./buggy_uaf
```

```
Stage 1 (malloc):
```

```
Pointer value : 0x503000000040  
Allocated size : 20 bytes (5 ints)
```

```
Stage 2 (realloc):
```

```
Pointer value : 0x504000000010  
Allocated size : 40 bytes (10 ints)
```

```
Stage 3 (free):
```

```
Pointer value freed : 0x504000000010  
Memory returned to heap allocator.
```

```
Demonstrating use-after-free:
```

```
Reading arr[0] after free...
```

```
=====
==521==ERROR: AddressSanitizer: heap-use-after-free on address 0x504000000010 at pc 0x55fbc1e356e5 bp
0x7ffed0b707e0 sp 0x7ffed0b707d0
READ of size 4 at 0x504000000010 thread T0
#0 0x55fbc1e356e4 in main /home/claude/exp2/buggy_uaf.c:54
#1 0x7f813122a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#2 0x7f813122a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#3 0x55fbc1e35224 in _start (/home/claude/exp2/buggy_uaf+0x1224) (BuildId:
a2d43286deb893f1a2ffa22bad909d6312c2ec)
```

```
0x504000000010 is located 0 bytes inside of 40-byte region [0x504000000010,0x504000000038)
freed by thread T0 here:
```

```
#0 0x7f81316fc4d8 in free ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:52
#1 0x55fbc1e355f0 in main /home/claude/exp2/buggy_uaf.c:44
#2 0x7f813122a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#3 0x7f813122a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#4 0x55fbc1e35224 in _start (/home/claude/exp2/buggy_uaf+0x1224) (BuildId:
a2d43286deb893f1a2ffa22bad909d6312c2ec)
```

```
previously allocated by thread T0 here:
```

```
#0 0x7f81316fc778 in realloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:85
#1 0x55fbc1e3547b in main /home/claude/exp2/buggy_uaf.c:29
#2 0x7f813122a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#3 0x7f813122a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId:
8e9fd827446c24067541ac5390e6f527fb5947bb)
#4 0x55fbc1e35224 in _start (/home/claude/exp2/buggy_uaf+0x1224) (BuildId:
a2d43286deb893f1a2ffa22bad909d6312c2ec)
```

```
SUMMARY: AddressSanitizer: heap-use-after-free /home/claude/exp2/buggy_uaf.c:54 in main
```

```
Shadow bytes around the buggy address:
```

```
0x503fffffd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x503fffffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x503fffffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x503ffffff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x503ffffff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x504000000000: fa fa[fd]fd fd fd fd fa fa fa fa fa fa fa fa fa
0x504000000080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x504000000100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x504000000180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x504000000200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x504000000280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
```

```
Shadow byte legend (one shadow byte represents 8 application bytes):
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
```

```
==521==ABORTING
```

4.4 Explanation of this error case

Stages 1-3 behave exactly as intended: 20 bytes are malloc'd, the block is grown to 40 bytes with realloc(), and both the pointer value and size are printed at each step. After free(arr) is called, the variable arr still holds the address of the now-released block (a dangling pointer) - the source code never sets it to NULL. When the program subsequently executes arr[0] to read a value, ASan detects a "heap-use-after-free" access at the exact freed address, and its report shows three stack traces: where the invalid access happened (main, line 54), where the block was freed (line 42), and where it was originally allocated by realloc (line 28) - making the root cause immediately traceable.

4.5 Security impact

Use-after-free is one of the most serious classes of memory-safety vulnerabilities in C/C++ software. Once memory is freed, the allocator is free to hand that same address to a completely different, attacker-influenced allocation. If the dangling pointer is used again afterwards: a read can leak sensitive data belonging to the new occupant of that memory; a write can corrupt that new object's data or internal pointers/vtables, which attackers routinely leverage to hijack control flow and achieve arbitrary code execution. UAF bugs are a leading root cause behind real-world browser, OS-kernel, and server exploits, which is why they are treated as high-severity, exploitable vulnerabilities (tracked as CWE-416) rather than 'just' crashes.

5. Corrected Version

The corrected program applies three fixes throughout the malloc -> realloc -> free lifecycle: (1) the return value of every malloc()/realloc() call is checked before the pointer is used; (2) newly (re)allocated memory is explicitly initialised before further use; and (3) the pointer is set to NULL immediately after free(), so that the class of use-after-free bug demonstrated above becomes structurally impossible through that variable (any accidental later use is now an obvious, fail-fast NULL check rather than silent heap corruption).

5.1 Code listing — fixed.c

```
/*
 * Experiment 2: Dynamic Memory Allocation, Reallocation, Deallocation
 * CORRECTED VERSION
 *
 * Fixes applied:
 * 1. Every malloc()/realloc() return value is checked before use.
 * 2. Memory is initialised immediately after (re)allocation.
 * 3. The pointer is set to NULL immediately after free(), so any
 *    accidental future use is an immediate, easily diagnosable
 *    NULL-pointer access instead of silent heap corruption.
 *
 * Compile with ASan (still useful to prove NO errors are reported):
 * gcc -fsanitize=address -g -O0 -o fixed fixed.c
 */

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;

    /* ---- Stage 1: malloc ---- */
    int *arr = (int *) malloc(n * sizeof(int));
    if (arr == NULL) { /* FIX 1: check return value */
        fprintf(stderr, "Error: malloc failed to allocate %zu bytes\n",
            n * sizeof(int));
        return EXIT_FAILURE;
    }
    for (int i = 0; i < n; i++) arr[i] = 0; /* FIX 2: initialise */

    printf("Stage 1 (malloc):\n");
    printf(" Pointer value : %p\n", (void *)arr);
    printf(" Allocated size : %zu bytes (%d ints)\n", n * sizeof(int), n);

    for (int i = 0; i < n; i++) arr[i] = i + 1;
    printf(" Initialised values: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n\n");

    /* ---- Stage 2: realloc ---- */
    int new_n = 10;
    int *tmp = (int *) realloc(arr, new_n * sizeof(int));
    if (tmp == NULL) { /* FIX 1: check return value */
        fprintf(stderr, "Error: realloc failed to grow to %zu bytes\n",
            new_n * sizeof(int));
        free(arr); /* original block still valid */
        arr = NULL;
        return EXIT_FAILURE;
    }
    arr = tmp;
    for (int i = n; i < new_n; i++) arr[i] = 0; /* FIX 2: initialise new region */

    printf("Stage 2 (realloc):\n");
    printf(" Pointer value : %p\n", (void *)arr);
    printf(" Allocated size : %zu bytes (%d ints)\n", new_n * sizeof(int), new_n);

    for (int i = n; i < new_n; i++) arr[i] = (i + 1) * 10;
    printf(" Values after realloc: ");
    for (int i = 0; i < new_n; i++) printf("%d ", arr[i]);
    printf("\n\n");

    /* ---- Stage 3: free ---- */
    free(arr);
    printf("Stage 3 (free):\n");
}
```

```

    printf(" Pointer value freed : %p\n", (void *)arr);

    arr = NULL; /* FIX 3: avoid dangling pointer */
    printf(" Pointer value after setting to NULL: %p\n", (void *)arr);
    printf(" Memory safely returned to heap allocator.\n\n");

    /* Any accidental future use is now caught immediately: */
    if (arr != NULL) {
        arr[0] = 999; /* unreachable, but shows the guard in place */
    } else {
        printf("Pointer is NULL - use-after-free is now impossible via this variable.\n");
    }

    return EXIT_SUCCESS;
}

```

5.2 Compilation and execution under ASan

```

$ gcc -fsanitize=address -g -O0 -o fixed fixed.c
$ ./fixed

```

```

Stage 1 (malloc):
  Pointer value   : 0x5030000000040
  Allocated size  : 20 bytes (5 ints)
  Initialised values: 1 2 3 4 5

Stage 2 (realloc):
  Pointer value   : 0x5040000000010
  Allocated size  : 40 bytes (10 ints)
  Values after realloc: 1 2 3 4 5 60 70 80 90 100

Stage 3 (free):
  Pointer value freed : 0x5040000000010
  Pointer value after setting to NULL: (nil)
  Memory safely returned to heap allocator.

Pointer is NULL - use-after-free is now impossible via this variable.
EXIT: 0

```

No ASan errors were reported for the corrected version, confirming that the allocation, resize, and deallocation lifecycle is now memory-safe.

6. Result

A C program demonstrating malloc(), realloc(), and free() was implemented, printing the pointer value and allocated size at each stage. Two intentional memory-safety defects were introduced and confirmed using AddressSanitizer:

Defect	ASan diagnostic	Root cause
(a) Unchecked malloc() -> NULL deref	SEGV on unknown address 0x000000000000	malloc() return value used without a NULL check
(b) Use-after-free	heap-use-after-free (READ) at freed address	pointer used after free() without being cleared

Both errors were explained in terms of their security impact (denial-of-service and potential arbitrary read/write from an unchecked NULL dereference; data leakage and potential control-flow hijack / arbitrary code execution from use-after-free). A corrected version of the program was then written that checks every allocation's return value, initialises memory after allocation, and sets the pointer to NULL immediately after free(); running this version under ASan produced no diagnostics, confirming the fix. All source files, the code, and captured ASan outputs were committed to a Git repository as required by the experiment.